অধ্যায় ২৭

Real-Time JavaScript Applications

WebSockets, Socket.IO, Event-Driven Architecture এবং Distributed Messaging

শেখার লক্ষ্য

এই অধ্যায় শেষে polling, Server-Sent Events (SSE), WebSocket, Socket.IO, event-driven design, event contracts, delivery semantics, reconnect, heartbeat, authorization, backpressure, Redis Pub/Sub ও distributed scaling-এর মূল ধারণা ব্যাখ্যা করতে পারবেন। TravelLive নামে একটি real-time booking event simulator তৈরি করবেন।

২৭.১ Real-Time Application কী?

একটি booking dashboard-এ একজন operator booking confirm করলে অন্য operator-এর screen-এ যত দ্রুত সম্ভব status update দেখা দরকার। শুধু page refresh করলে তা হয় না। Real-time application সাধারণত server থেকে client-এ নতুন event পৌঁছে দেয়; তবে network delay, connection drop ও duplicate delivery বিবেচনায় নিতে হয়। **Real-time মানে guaranteed instant নয়**—এটি low-latency update-এর একটি engineering goal।

Use cases: booking status updates, live seat availability, chat, collaborative dashboards, notifications, delivery tracking এবং support agent presence।

২৭.২ Polling, Long Polling, SSE এবং WebSocket

Technique	Communication	Suitable use	Limitations
Short polling	Repeated HTTP request	Rare updates, simple clients	Unnecessary requests, delayed updates
Long polling	Server holds request until data/timeout	Fallback	Connection churn
SSE	Server → browser, HTTP stream	Notifications, status feed	One-way, text/event format
WebSocket	Bidirectional persistent connection	Chat, live collaboration	Connection lifecycle, scaling
Socket.IO	Event-oriented library with transport abstraction	Rooms, reconnect support	Socket.IO protocol, not raw WebSocket-compatible

Socket.IO client দিয়ে arbitrary raw WebSocket server-এ সরাসরি connect করা যায় না; উভয় পাশে compatible protocol দরকার। SSE-তে `EventSource` ব্যবহার করা যায়; authenticated cross-origin requests-এর cookie/CORS আচরণ বুঝতে হবে।

২৭.৩ WebSocket Lifecycle

Browser একটি HTTP Upgrade handshake দিয়ে WebSocket সংযোগ শুরু করতে পারে। সংযোগ সফল হলে client ও server দুই দিকেই message পাঠাতে পারে। `open`, `message`, `error`, `close` lifecycle events handle করতে হয়। Browser-side উদাহরণ:

```javascript
const socket = new WebSocket('wss://example.com/live');
socket.addEventListener('open', () => console.log('Connected'));
socket.addEventListener('message', (event) => {
  try { const update = JSON.parse(event.data); console.log(update); }
  catch { console.warn('Invalid server message'); }
});
socket.addEventListener('close', () => console.log('Disconnected'));
```

Production-এ `wss://`, authenticated session, allowed origins, per-message validation, message-size limit এবং rate limiting জরুরি। শুধু client-এ validation থাকলে নিরাপত্তা নিশ্চিত হয় না।

২৭.৪ Socket.IO Events এবং Rooms

Socket.IO server-এ booking-specific room-এ authorized clients যুক্ত করা যায়। নিচের snippet একটি **illustrative server design**, bundle-এর tested implementation নয়:

```javascript
io.use(async (socket, next) => {
  try {
    socket.data.user = await authenticateSession(socket.request);
    next();
  } catch { next(new Error('UNAUTHORIZED')); }
});

io.on('connection', (socket) => {
  socket.on('booking:subscribe', async ({ bookingId }, ack) => {
    if (typeof bookingId !== 'string') return ack?.({ ok: false });
    if (!await canViewBooking(socket.data.user.id, bookingId)) {
      return ack?.({ ok: false });
    }
    await socket.join(`booking:${bookingId}`);
    ack?.({ ok: true });
  });
});
```

Important: Room name গোপন রাখলেই authorization হয় না। প্রতিটি subscribe ও mutation server-side অনুমতি যাচাই করতে হবে। Room membership connection-specific; reconnect হলে আবার subscribe করতে হতে পারে।

২৭.৫ Event Contract ও Versioning

অর্থপূর্ণ event envelope ব্যবহার করলে client সহজে handle করতে পারে:

```json
{
  "eventId": "evt-102",
  "type": "booking.status.changed",
  "version": 1,
  "bookingId": "bk-22",
  "sequence": 8,
  "occurredAt": "2026-10-11T00:00:00.000Z",
  "payload": { "status": "CONFIRMED" }
}
```

`eventId` duplicate detection-এ, `sequence` per-booking ordering-এ, `version` schema evolution-এ কাজে লাগে। Event payload-এ password, access token, ব্যক্তিগত তথ্য বা unnecessary financial details পাঠাবেন না।

২৭.৬ Event-Driven Architecture

Command (যেমন `ConfirmBooking`) একটি action-এর অনুরোধ; Event (`BookingConfirmed`) ঘটে যাওয়া fact। Command validation ও authorization-এর পর state change হয়, তারপর event publish করা হয়। একই event দিয়ে notification, analytics এবং audit trail update করা যায়।

```
Client Command → API/Auth → Booking Service → Database Transaction
                                                    ↓
                                              Outbox Record
                                                    ↓
                                            Event Publisher
                                                    ↓
                                        Broker / Socket Gateway
                                                    ↓
                                          Authorized Clients
```

Transactional outbox pattern-এ business update ও outbox event একই database transaction-এ commit হয়; background worker পরে publish করে। এটি database commit সফল কিন্তু publish ব্যর্থ হওয়ার gap কমায়। Broker duplicate delivery করতে পারে, তাই consumers idempotent হওয়া উচিত।

২৭.৭ Delivery Semantics এবং Ordering

- **At-most-once:** Event হারাতে পারে, কিন্তু retry করে duplicate তৈরি হয় না।
- **At-least-once:** Retry-এর কারণে duplicate আসতে পারে; idempotency দরকার।
- **Exactly-once effect:** End-to-end বাস্তবায়ন কঠিন; business operation-এ deduplication, transactions ও unique constraints ব্যবহার করুন।
- **Ordering:** Global ordering ধরে নেবেন না। প্রয়োজন হলে per-aggregate sequence রাখুন এবং gap হলে server snapshot fetch করুন।

Client reconnect-এর পর সর্বশেষ `sequence` পাঠিয়ে missed events replay চাইতে পারে। Replay history না থাকলে authoritative API থেকে full snapshot আনতে হবে।

২৭.৮ Reconnect, Heartbeat এবং Backpressure

Connection drop হওয়া স্বাভাবিক। Exponential backoff + jitter দিয়ে reconnect করতে হবে; infinite immediate retry server overload করে। Heartbeat/ping-pong দিয়ে dead connection শনাক্ত করা যায়। Slow client-এ unlimited queue রাখলে memory বাড়ে; bounded buffer, coalescing অথবা disconnect policy প্রয়োজন।

```javascript
function reconnectDelay(attempt, random = Math.random) {
  const capped = Math.min(30_000, 500 * 2 ** Math.min(attempt, 6));
  return Math.floor(capped * (0.75 + random() * 0.5));
}
```

এই উদাহরণে `random` inject করা হয়েছে, যাতে deterministic tests লেখা যায়।

২৭.৯ Redis Pub/Sub এবং Multi-Instance Scaling

একাধিক Socket.IO server থাকলে একই room-এর client বিভিন্ন instance-এ থাকতে পারে। Redis adapter ব্যবহার করে event fan-out সমন্বয় করা যায়। তবে Redis Pub/Sub **durable queue নয়**—offline subscriber-এর জন্য history ধরে রাখে না। Reliable replay দরকার হলে Redis Streams, Kafka বা persistent event store বিবেচনা করুন। Load balancer, sticky sessions, transport choice এবং shared authorization state-ও পরিকল্পনায় রাখতে হবে।

২৭.১০ Security Checklist

১. TLS (`wss://`) ও origin policy যাচাই করুন। ২. Session authentication এবং per-resource authorization দিন। ৩. Client message schema, size ও rate limit enforce করুন। ৪. Event data minimization করুন। ৫. Logout/revocation-এর পর unauthorized subscription বন্ধ করুন। ৬. Observability-তে connection count, publish lag, failed authorization, dropped events ও queue depth মাপুন।

২৭.১১ Practical Project — TravelLive

এই অধ্যায়ের `source-code/chapter-27/`-এ একটি **dependency-free, in-memory real-time event simulator** রয়েছে। এতে booking creation, status transition, room subscriptions, sequence-based replay, duplicate event suppression, unauthorized room access prevention, capped history এবং reconnect delay calculation আছে। Browser UI ও actual network socket নয়; core event architecture শেখার জন্য intentionally ছোট রাখা হয়েছে।

Project Structure

```
chapter-27/
├── package.json
├── live-core.mjs
├── live-core.test.mjs
├── demo.mjs
└── README.md
```

Run

```
TERMINAL
cd source-code/chapter-27
node --test
node demo.mjs
```

Business rule: `PENDING → CONFIRMED/CANCELLED`; `CONFIRMED → CANCELLED`; `CANCELLED` terminal। User কেবল নিজের booking-এর event দেখতে পারবেন। Subscription authorization শুধু subscribe-time-এ নয়, event dispatch-এর সময়ও পরীক্ষা করা হয়। এই simulator single-process; multi-process durable replay, network reconnection, DB persistence ও production security দেয় না।

২৭.১২ Debugging এবং Common Mistakes

Mistake 1: Socket connected মানেই user authorized—ভুল। **Mistake 2:** `emit` সফল মানেই client processed—ভুল; acknowledgments এবং retry দরকার হতে পারে। **Mistake 3:** Reconnect-এর পর event loss নেই—ভুল; replay/snapshot strategy লাগবে। **Mistake 4:** Event ID ছাড়াই repeated updates apply করা—duplicate side effects হতে পারে। **Mistake 5:** Redis Pub/Sub-কে persistent message queue ভাবা—ভুল।

২৭.১৩ Exercises

১. Polling ও SSE-এর একটি tradeoff লিখুন।
২. Socket.IO কেন raw WebSocket-এর সঙ্গে wire-compatible নয়?
৩. Booking-specific room-এর authorization কোথায় করবেন?
৪. `eventId` ও `sequence`-এর আলাদা কাজ কী?
৫. At-least-once delivery-তে idempotency কেন জরুরি?
৬. Client reconnect হলে gap recovery কীভাবে করবেন?
৭. Redis Pub/Sub-এর durability limitation ব্যাখ্যা করুন।
৮. Transactional outbox কী সমস্যা সমাধান করে?
৯. Slow client-এর backpressure policy ডিজাইন করুন।
১০. TravelLive-এ booking owner ছাড়া অন্য user-এর subscription reject করার test লিখুন।

২৭.১৪ Sample Solutions

১. Polling সহজ কিন্তু idle request বেশি; SSE server-initiated updates efficiently পাঠায়, তবে একমুখী।
২. Socket.IO নিজস্ব framing, handshake ও event protocol ব্যবহার করে।
৩. Server-side authenticated user ও booking ownership যাচাই করে।
৪. `eventId` deduplication, `sequence` per-booking ordering/gap detection।
৫. Retry duplicate event তৈরি করতে পারে; idempotent consumer একই action দ্বিতীয়বার করবে না।
৬. Last-seen sequence পাঠিয়ে replay; gap unrecoverable হলে full snapshot fetch।
৭. Subscriber offline থাকলে Pub/Sub message ধরে রাখে না।
৮. Database commit ও event publication-এর dual-write inconsistency কমায়।
৯. Bounded queue, coalescing, slow-client disconnect এবং metrics।
১০. `assert.throws(() => live.subscribe('other-user', bookingId, fn), /FORBIDDEN/)`।

২৭.১৫ MCQ

১. Bidirectional persistent transport কোনটি? (A) CSS (B) WebSocket (C) JSON (D) localStorage
২. SSE-এর direction? (A) Client only (B) Server → Client (C) Both (D) None
৩. Room subscription authorization কে করবে? (A) Browser (B) Server (C) CSS (D) URL alone
৪. Duplicate event চিনতে কোনটি সহায়ক? (A) eventId (B) font-size (C) margin (D) hostname
৫. Per-booking ordering-এর জন্য? (A) random sort (B) sequence (C) CSS order (D) cookie name
৬. Redis Pub/Sub offline history রাখে? (A) Always (B) No (C) 30 days (D) One year
৭. Transactional outbox কোথায় লেখা হয়? (A) Same DB transaction (B) CSS (C) Browser only (D) CDN
৮. Reconnect strategy? (A) Immediate infinite loop (B) Exponential backoff with jitter (C) No retry (D) Delete data
৯. Backpressure-এর একটি সমাধান? (A) Unlimited queue (B) Bounded buffer (C) Disable validation (D) Ignore memory
১০. At-least-once delivery-তে কী ঘটতে পারে? (A) Duplicate (B) No network (C) Guaranteed order (D) Zero latency

Answer key: ১-B, ২-B, ৩-B, ৪-A, ৫-B, ৬-B, ৭-A, ৮-B, ৯-B, ১০-A।

২৭.১৬ Chapter Summary

Real-time system তৈরি মানে শুধু socket connection নয়। Correctness-এর জন্য authorization, event schema, replay, idempotency, failure handling এবং scaling strategy দরকার। Chapter 28-এ এই ধরনের application Docker, CI/CD, Cloud ও Monitoring-এর মাধ্যমে deploy করার পদ্ধতি শিখব।